Análisis y Diseño de Algoritmos I

Valeria Bengolea
Dpto. de Computación - FCEFQyN - UNRC

Departamento de Computación
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Universidad Nacional de Río Cuarto

Introducción a Haskell

Material basado en slides realizadas por el Dr. Pablo Castro, Introducción a los Algoritmos I, Departamento de Computación, FCEFQyN, UNRC, Argentina

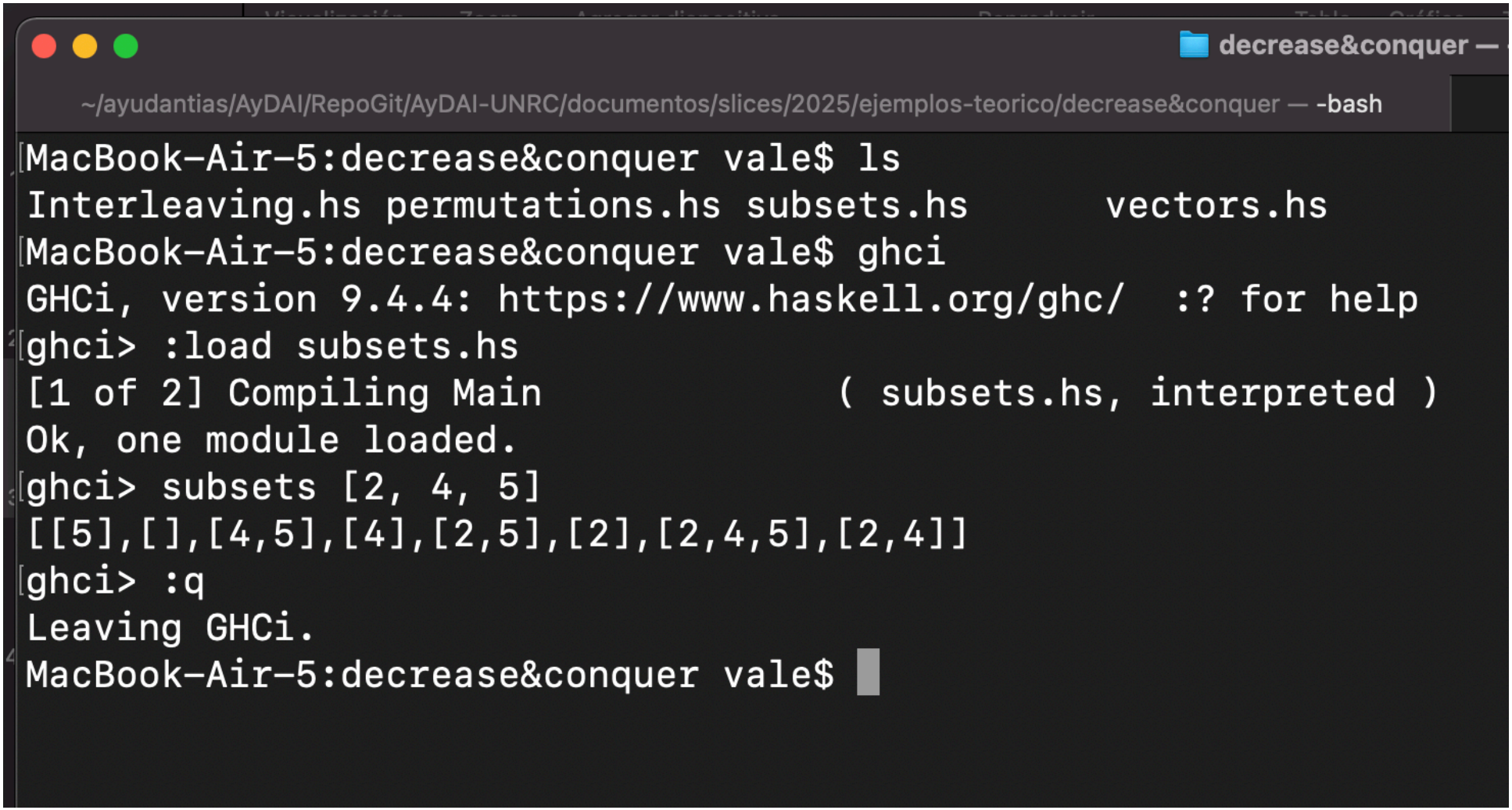
El Lenguaje Haskell

Haskell fue introducido en 1987 con el objetivo de introducir un lenguaje funcional moderno

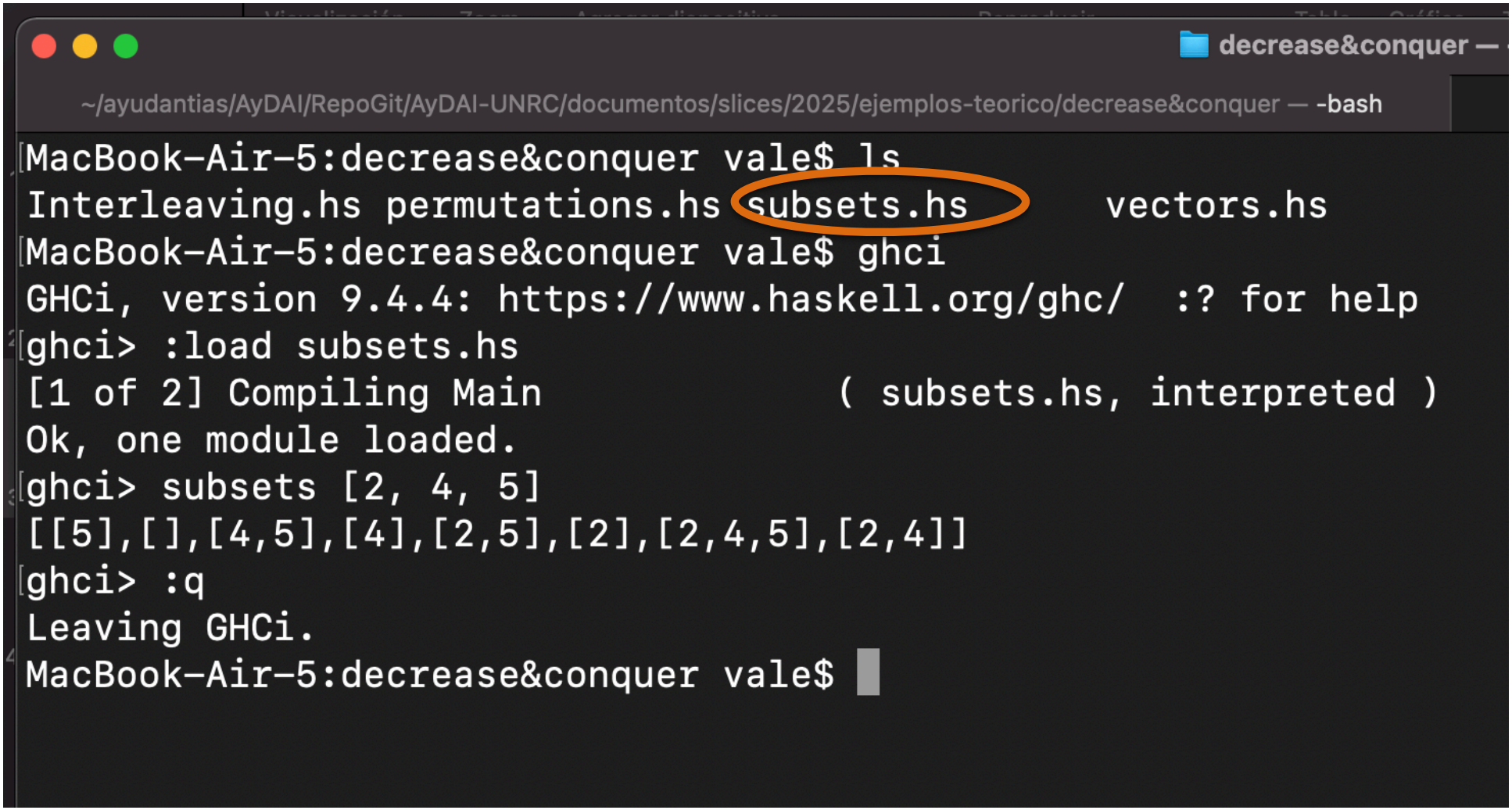
- Hugs, es un interprete de Haskell muy usado, se puede obtener en www.haskell.org/hugs
- Glasgow Haskell, es un compilador para Haskell se puede obtener en http://www.haskell.org/ghc/.

ghc es más completo y también tiene **un interprete ghci**

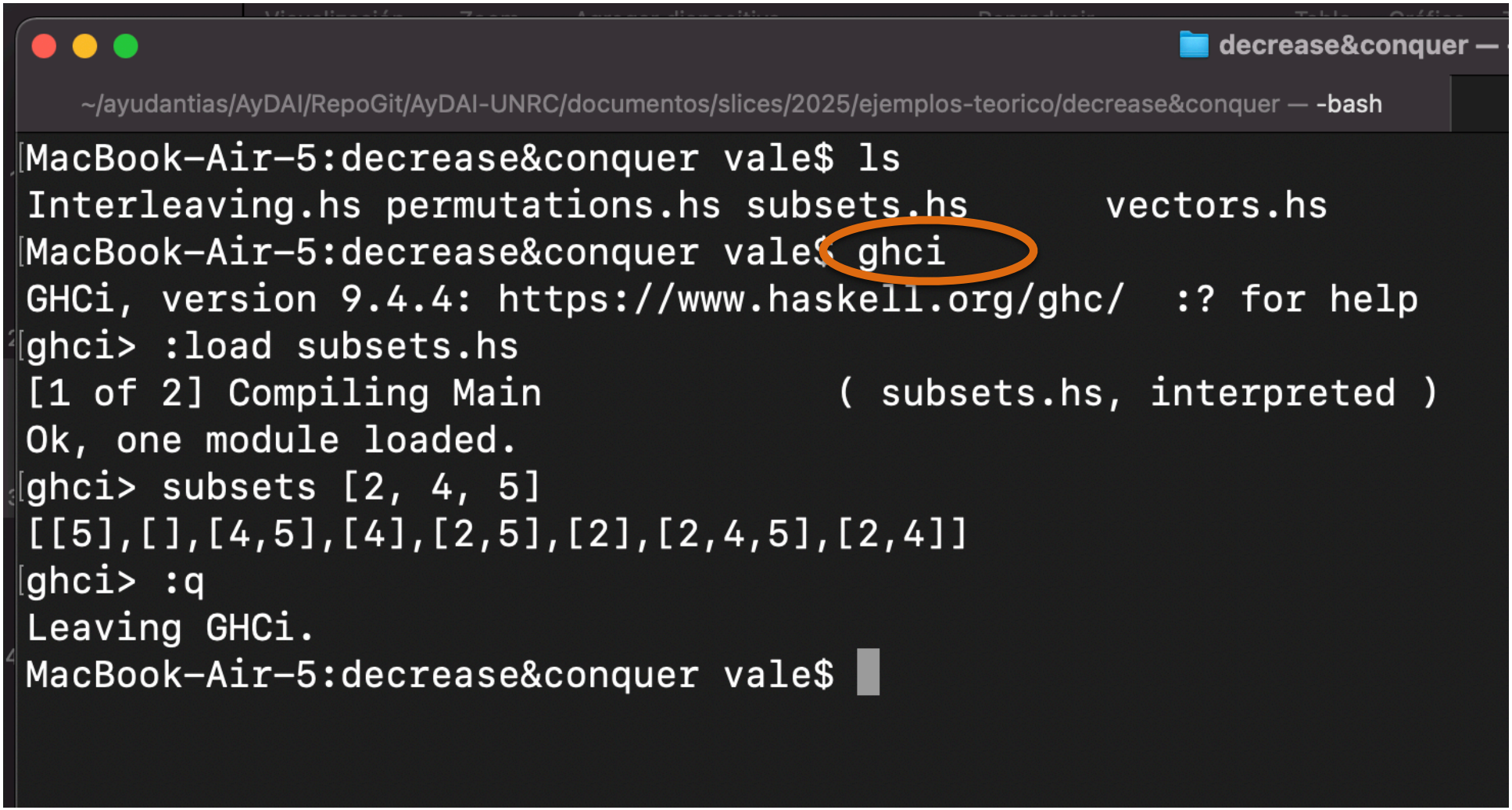
El Lenguaje Haskell



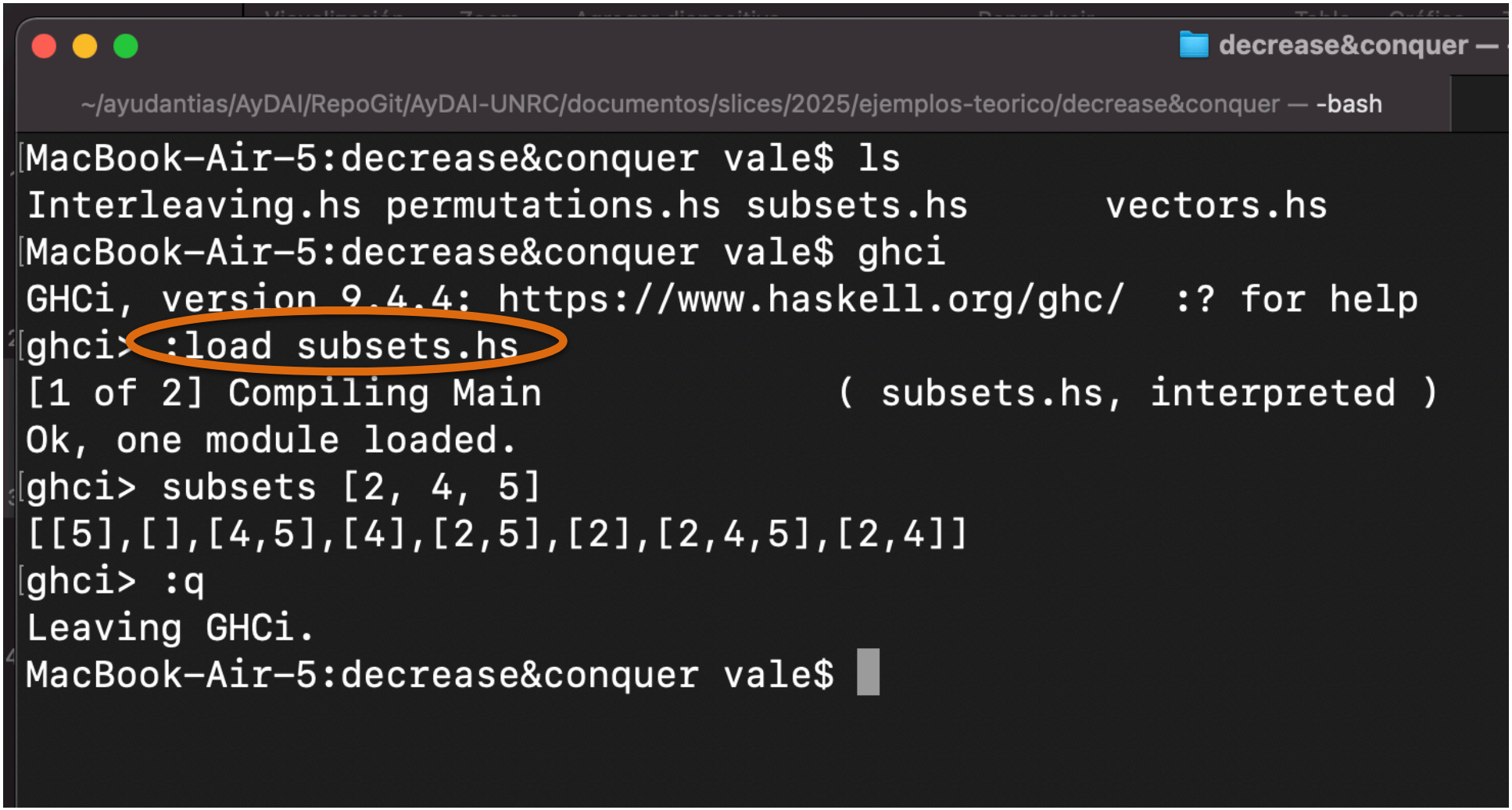
El Lenguaje Haskell



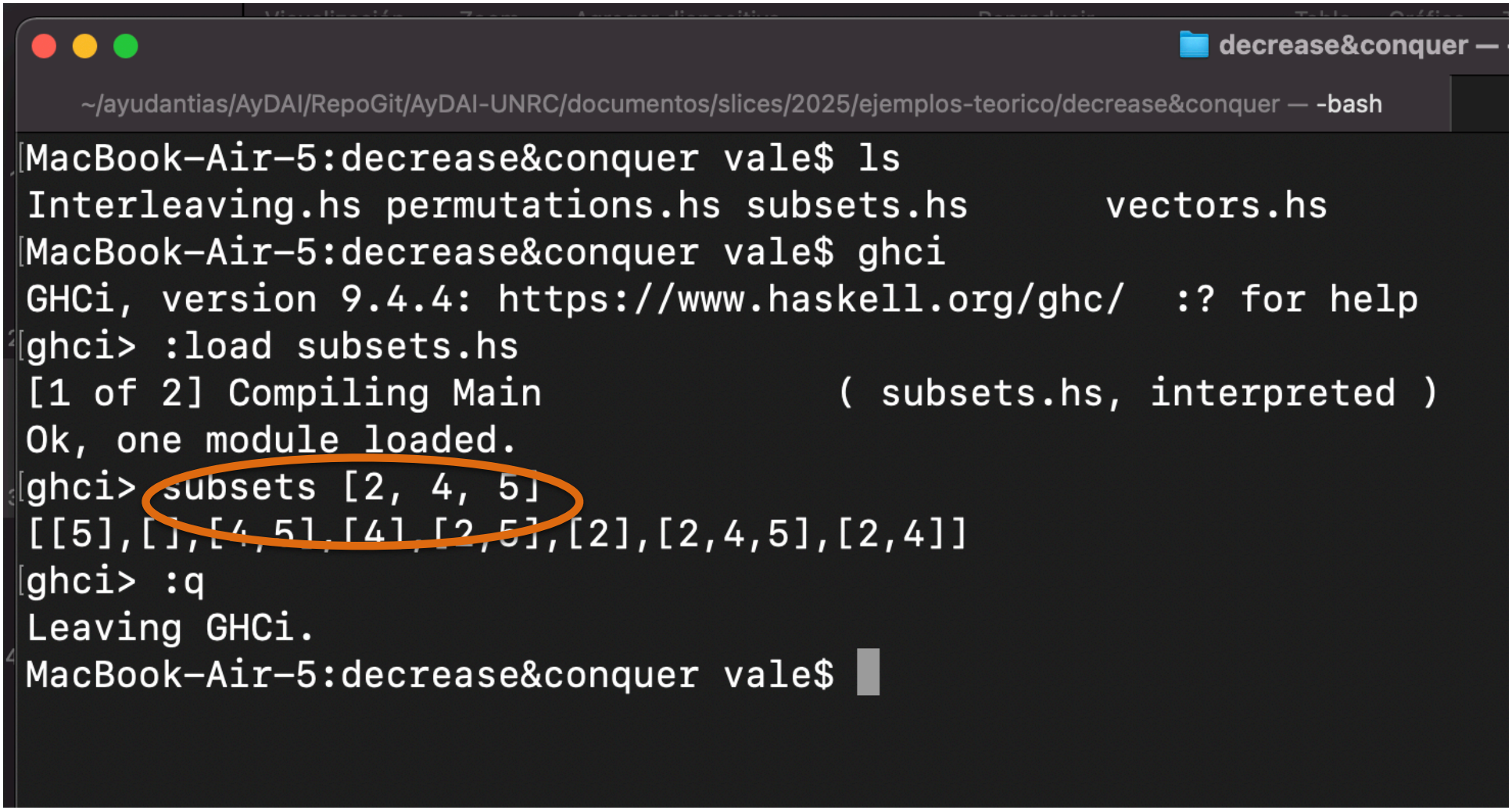
El Lenguaje Haskell



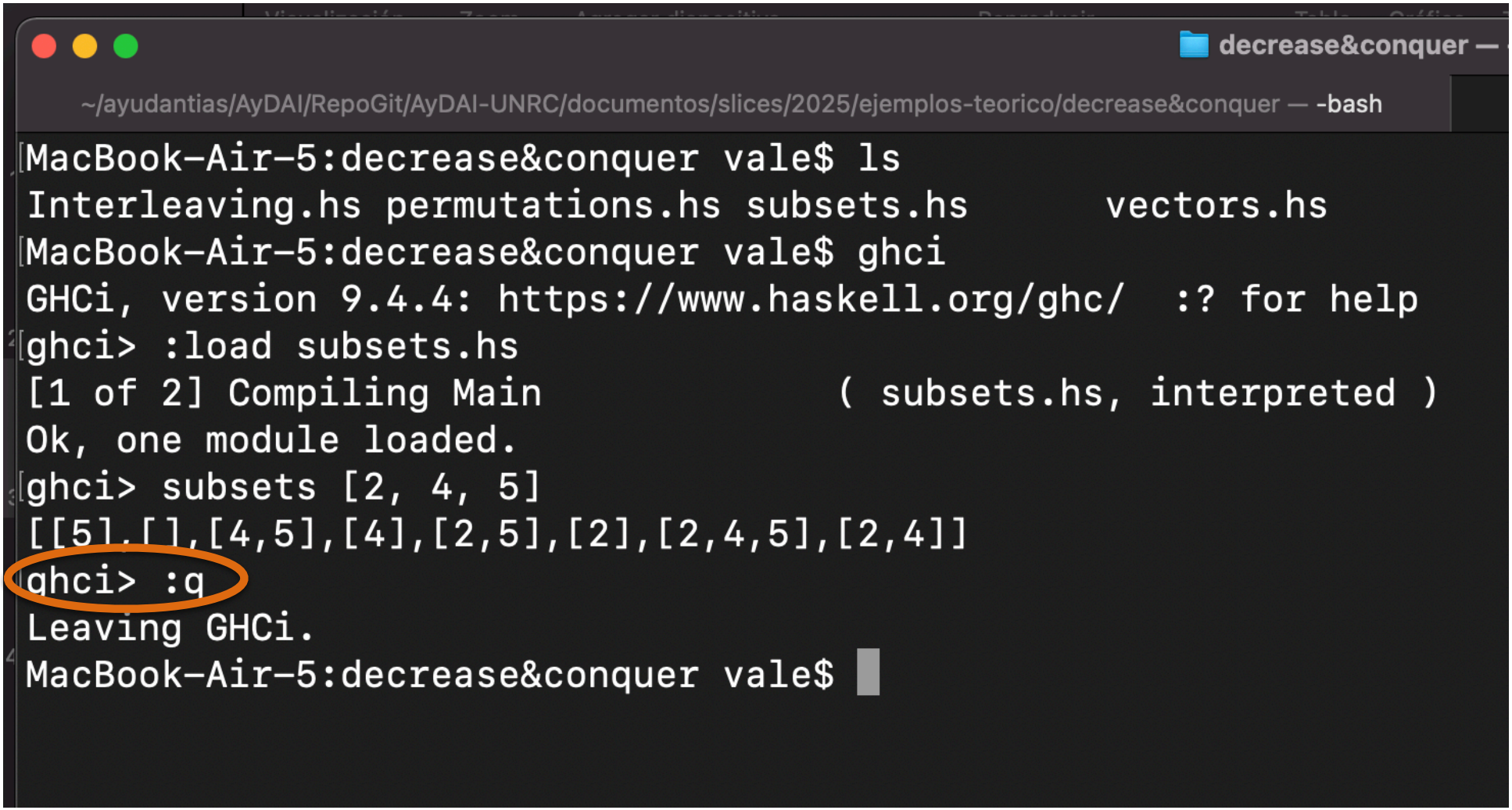
El Lenguaje Haskell



El Lenguaje Haskell



El Lenguaje Haskell



Tipos Básicos de Haskell

Haskell tiene un conjunto rico de tipos básicos de datos

- **Booleanos**, tipos de booleanos con las operaciones lógicas,
- **Int**: Enteros de precisión fija,
- **Char**: ‘a’, ‘b’, ‘c’, etc
- **Integer**: Enteros de precisión variable,
- **Float**: Números reales.

Escribimos: `E::T` Cuando `E` es de tipo `T`

El Tipo Bool

El tipo `Bool` tiene dos valores `true` y `false`, y las siguientes operaciones:

- `&& :: Bool->Bool->Bool`
- `|| :: Bool->Bool->Bool`
- `not :: Bool->Bool`

Cualquier función: `f::a->Bool` es llamado predicado, y puede utilizarse con estas operaciones.

`==::a->a->Bool`

La igualdad es la más conocida

El Tipo Char

El tipo `char` contiene los valores `'a','b','c',` ... etc, y las siguientes funciones:

- `ord::Char -> Int` convierte caracteres a enteros.
- `chr::Int -> Char` convierte enteros a caracteres.

Los Strings se modelan como una lista de chars.

Sistema de Números

Haskell tiene varios tipos numéricos:

- `Int`, enteros con precisión limitada 32/64 bits.
- `Integer`, enteros con precision variable (el tamaño crece según sea necesario)
- `Float`, reales 3.14159
- `Double`, reales con doble precisión.

Tenemos funciones de conversión entre ellos, por ejemplo:

`fromInteger` ← Permite convertir un entero a otro tipo numérico

Tuplas

Usando los tipos básicos podemos construir tuplas y listas.
Dados tipos `a` y `b`:

`(a,b)` < El tipo de pares de `a` y `b`

Por ejemplo: `(True,1)::(Bool,Int)`

Operaciones:

`fst::(a,b)->a` < Devuelve el primer componente

`snd::(a,b)->b` < Devuelve el segundo componente

Listas

Dado un tipo cualquiera `a`

`[a]` Es el tipo de las listas de tipo `a`

Donde:

- `[]` es la lista vacía
- `x:xs` es la lista con x a la cabeza y luego xs a la cola

Todos los elementos de la lista son del mismo tipo.

Funciones sobre Listas

Algunas funciones útiles sobre listas:

- `head::[a]->a`, devuelve la cabeza de la lista.
- `last::[a]->a`, devuelve el último elemento.
- `tail::[a]->[a]`, devuelve la cola de la lista.
- `++::[a]->[a]->[a]`, concatena dos listas.

Funciones

Dados dos tipos a y b:

`a->b` — Es el tipo de las funciones de `a` en `b`

Por ejemplo:

`Not:Bool->Bool` — Función de `Bool` en `Bool`

Las funciones de alto orden son aquellas que toman como parámetros funciones o retornan funciones

`.:(b->c)->(a->b)->(a->c)` — Define la composición de funciones

Definición de Funciones

Una función se puede definir por casos:

```
sign :: Int->Int
sign x | x>=0 =1
       | x<0  =-1
```

También por pattern matching:

```
take :: Int->[a]->[a]
take 0 xs = []
take n [] = []
take n (x:xs) = x:take (n-1) xs
```

Toma los primeros n elementos de una lista

Se pueden usar los constructores de los tipos

Polimorfismo

Capacidad de una función o expresión de trabajar con valores de distintos tipos.

Consideremos la siguiente función:

```
drop::Int->[a]->[a]
drop n [] = []
drop 0 xs = xs
drop n (x:xs) = drop (n-1) xs
```

Descarta los primeros n elementos de la lista

En `[a]`, `a` puede ser cualquier tipo, se llama variable de tipos, y `drop` se dice que es polimórfica.

foldr

Consideremos la función:

```
sum [] = 0
sum (x:xs) = x+(sum xs)
```

Si ejecutamos esta función para [1,..,n] obtenemos:

```
1+(2+(3+(...+n))
```

Asocia a la derecha

De la misma forma podríamos reemplazar el + por *

```
1*(2*(3*(...*n))
```

foldr

Podemos generalizar esto para una operación Op

Para `[X0,X1,…,Xn]` calculamos

$$X_0\ Op\ (X_1\ Op\ (\ldots Op\ X_n))$$

Captura un **patrón** de recursión

Esto se define por medio de la función `foldr`

foldr

`foldr`

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z [] = z
foldr f z (x:xs) = f x (foldr f z xs)
```

z es el elemento que devolvemos en el caso []

Ejemplo: `foldr (&&) True [X0,X1,X2] = X0 &&(X1&&(X2 && True))`

Ejemplos

- `sum xs  = foldr (+) 0 xs`
- `mult xs = foldr (*) 1 xs`
- `conj xs = foldr (&&) true xs`
- `disj xs = foldr (||) false xs`

Map

Map, dada una función y una lista aplica esa función a cada elemento de la lista

```
map :: (a -> b) -> [a] -> [b]
map f [] = []
map f (x:xs) = f x : (map f xs)
```

Es decir:

map f $[X_0, X_1, X_2, .., X_n]$ = [f X_0, f X_1, …, f X_n]

Por ejemplo:

```
map length ["hola", "haskell", "curso"] = [4,7,5]
```

```
map not [True, False, True, False] = [False, True, False, True]
```

Filter

Dada una lista filtra los elementos de la lista usando un predicado:

```
filter :: (a->Bool)->[a]->[a]
filter f [] = []
filter f (x:xs)
                | f x =x:(filter f xs)
                | Otherwise = filter f xs
```

Por ejemplo:

Puede funcionar para listas infinitas

```
take 5 (filter even [1..])
```

```
filter (> 10) [3,12,7,15,2,20] = [12,15,20]
```

Listas por Comprensión

En conjuntos podemos hacer:

$$\{2 * x \mid x \in \{0, 1, 2, 3, 4\}\}$$

En Haskell tenemos listas por comprensión, por ejemplo:

```
[2*x  |  x<-[0,1,2,3,4]]
```

Generador

Retorna la lista:

```
[0,2,4,6,8]
```

Listas por Comprensión

Podemos tener muchos generadores:

```
[(x,y) | x<-[0,1,2],y<-[2,3,4]]
```

Devuelve:

```
[(0,2),(0,3),(0,4),(1,2),(1,3),(1,4),(2,2),(2,3),(2,4)]
```

Podemos utilizar guardas para filtrar elementos:

```
[x | x<-[0..], even x]
```

Devuelve:

```
[0,2,4,6,8,10,…]
```

Ejemplos:

Remueve todos los caracteres que no son mayúsculas de una cadena

```
removeNonUppercase st = [c | c <-st, c `elem` ['A'..'Z']]
```

```
> removeNonUppercase “Hola"
“H”
```

```
triplasPit = [(a,b,c) | c<-[1..], b<-[1..c], a<-[1..b], c^2 == a^2+b^2]
```

```
> triplasPit
(3,4,5), (6,8,10), (5,12,13), …
> take 5 triplasPit
[(3,4,5),(6,8,10),(5,12,13),(9,12,15),(8,15,17)]
```

Devuelve todas las triplas pitagóricas: $x^2 + y^2 = z^2$

Ejemplos:

```
Divisores n = [x | x<-[1..n], n `mod` x == 0]
```

```
> divisores 4
  [1,2,4]
```

Devuelve todos los divisores de un número

```
esPrimo n = (divisores n) == [1,n]
```

Decide si un número es primo o no

Declarando Tipos

Podemos definir sinónimos de tipos con el constructor `type`:

```
type String = [Char]
```

Es la definición de String en Haskell

También por ejemplo:

```
type Pos =(Int,Int)
```

Posiciones en un tablero

Y podemos usar este tipo nuevo:

```
type Board =[Pos]
```

Un tablero es una lista de posiciones

Tipos Nuevos

Podemos definir tipos con nuevos valores:

```
data Color = Rojo | Verde | Azul
```

Y se pueden definir tipos inductivos

```
data Nat = Zero|Succ Nat
```

Definen los naturales por medio de dos constructores:

`Zero::Nat` y `Succ::Nat->Nat`

Cuyos valores son:

```
Zero, Succ Zero, Succ (Succ Zero), …
```

Usando Notación de Registros

Cuando definimos tipos que tienen varios “campos” de información podemos usar la notación registro.

Por ejemplo, en vez de:

```
data Person = Person String String Int Float String String deriving (Show)
```

Podemos usar:

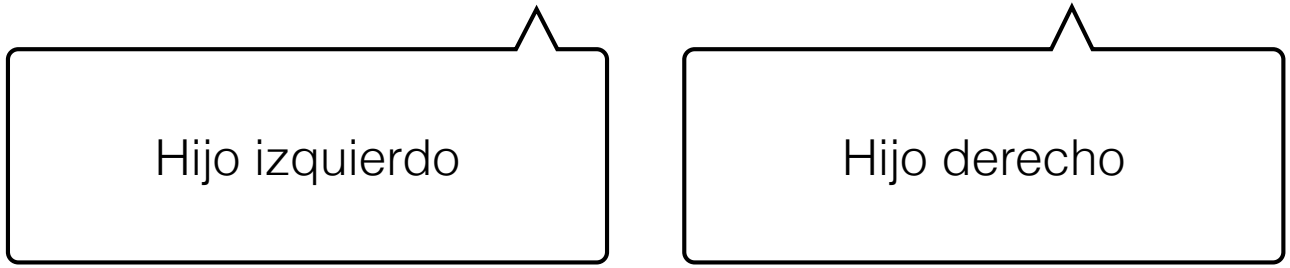
```
data Person = Person { firstName :: String
                     , lastName :: String
                     , age :: Int
                     , height :: Float
                     , phoneNumber :: String
                     , flavor :: String
                     } deriving (Show)
```

Nos da una función para acceder a cada campo.

Árboles Binarios

Los árboles binarios son un tipo de datos muy útil:

```
Data BinTree a = Nil | Node (BinTree a) a (BinTree a)
```



Un ejemplo de función sobre árboles:

```
size:: BinTree a->Int
size Nil = 0
size (Node hi r hd) = 1 + size hi + size hd
```